VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



CO3010 - Microprocessors-Microcontrollers

LAB REPORT

Lab 2 - Timer Interrupt and LED Scanning

Advisor(s): Phan Văn Sỹ
Student(s): Nguyễn Đức Phi Long ID 2452689

HO CHI MINH CITY, OCTOBER 2026

Contents

1 Github **4**

2 Exercise and Report **4**

2.1 Exercise 1 4
2.2 Exercise 2 9
2.3 Exercise 3 12
2.4 Exercise 4 14
2.5 Exercise 5 15
2.6 Exercise 6 16
2.7 Exercise 7 19
2.8 Exercise 8 19
2.9 Exercise 9 22
2.10 Exercise 10 30

1 Github

Link for the every exercises in Lab 1: Link

2 Exercise and Report

2.1 Exercise 1

The first exercise show how to interface for multiple seven segment LEDs to STM32F103C6 micro-controller (MCU). Seven segment displays are common anode type, meaning that the anode of all LEDs are tied together as a single terminal and cathodes are left alone as individual pins.

In order to save the resource of the MCU, individual cathode pins from all the seven segment LEDs are connected together, and connect to 7 pins of the MCU. These pins are popular known as the **signal pins**. Meanwhile, the anode pin of each seven segment LEDs are controlled under a power enabling circuit, for instance, an PNP transistor. At a given time, only one seven segment LED is turned on. However, if the delay is small enough, it seems that all LEDs are enabling.

Implement the circuit simulation in Proteus with two 7-SEGMENT LEDs as following:

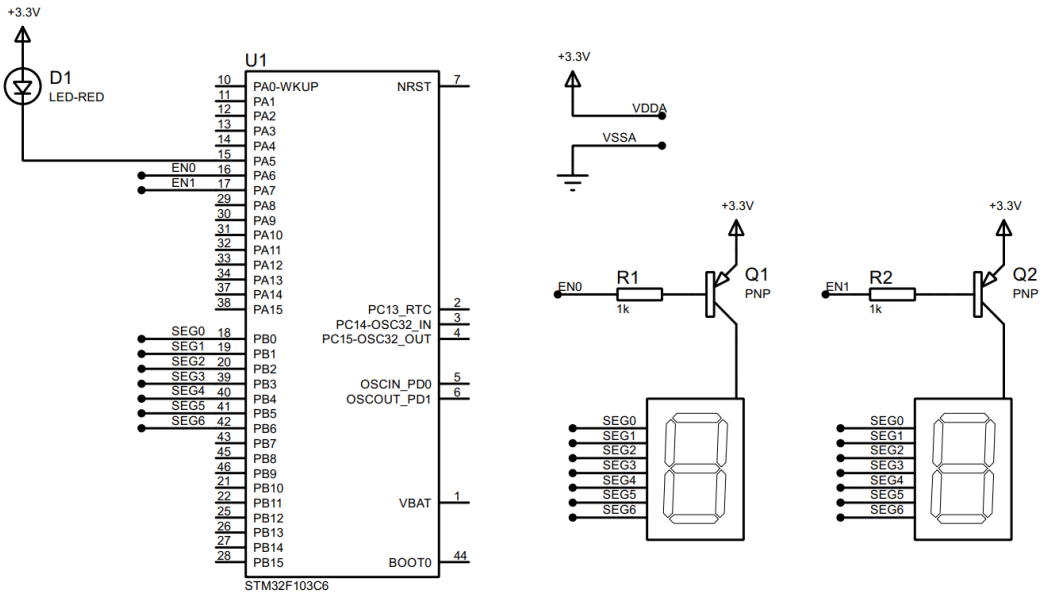


Figure 2.1: *Simulation schematic in Proteus*

Components used in the schematic are listed bellow:

- 7SEG-COM-ANODE (connected from PB0 to PB6)
- LED-RED
- PNP
- RES
- STM32F103C6

Students are proposed to use the function **display7SEG(int num)** in the Lab 1 in this exercise. Implement the source code in the interrupt callback function to display number **"1"** on the first seven segment and number **"2"** for second one. The switching time between 2 LEDs is half of second.

Report 1: Capture your schematic from Proteus and show in the report.

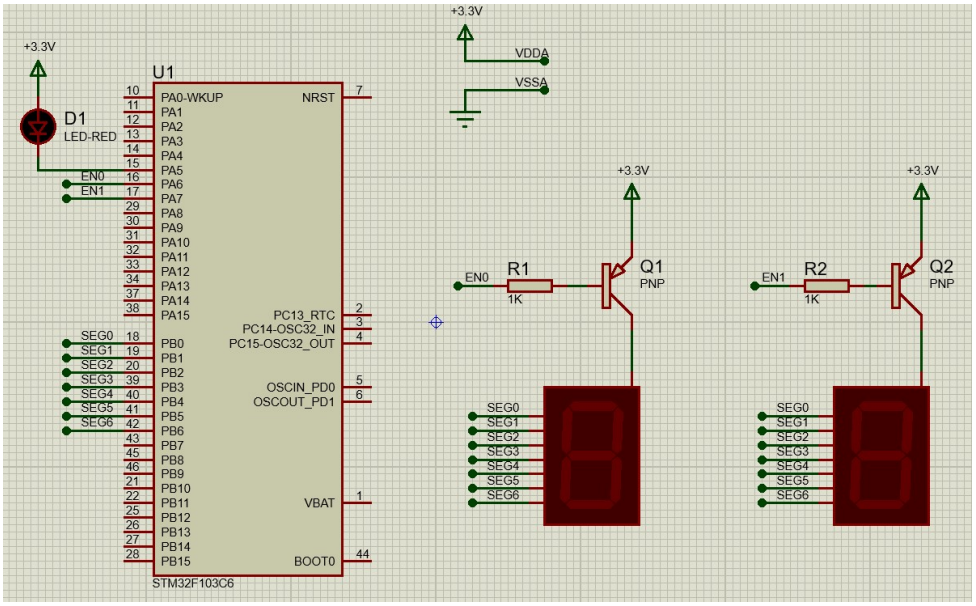


Figure 2.2: *Proteus schematic for two 7-segment LEDs*

Report 2: Present your source code in the **HAL_TIM_PeriodElapsedCallback** function.

- Before implementing function `void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)`, we need to initialize essential variables and functions from Lab 1.

Note: This should be written in `/* USER CODE BEGIN 0 */` block as we define them globally and easier to look out afterward.

Listing 2.1: Initial implementation for Exercise 01

```
/* USER CODE BEGIN 0 */
typedef struct
{
    GPIO_TypeDef *port;
    uint16_t pin;
} SegPin;


SegPin segments[] =
{
    { SEG0_GPIO_Port, SEG0_Pin },
    { SEG1_GPIO_Port, SEG1_Pin },
    { SEG2_GPIO_Port, SEG2_Pin },
    { SEG3_GPIO_Port, SEG3_Pin },
    { SEG4_GPIO_Port, SEG4_Pin },
    { SEG5_GPIO_Port, SEG5_Pin },
    { SEG6_GPIO_Port, SEG6_Pin },
};


GPIO_PinState digitPatterns[10][7] =
{
    { RESET, RESET, RESET, RESET, RESET, RESET, SET   },  //
      0
    { SET,   RESET, RESET, SET,   SET,   SET,   SET   },  //
      1
    { RESET, RESET, SET,   RESET, RESET, SET,   RESET },  //
      2
    { RESET, RESET, RESET, RESET, SET,   SET,   RESET },  //
      3
    { SET,   RESET, RESET, SET,   SET,   RESET, RESET },  //
      4
```

```
28      { RESET, SET,   RESET, RESET, SET,   RESET, RESET }, //
            5
29      { RESET, SET,   RESET, RESET, RESET, RESET, RESET }, //
            6
30      { RESET, RESET, RESET, SET,   SET,   SET,   SET   }, //
            7
31      { RESET, RESET, RESET, RESET, RESET, RESET, RESET }, //
            8
32      { RESET, RESET, RESET, RESET, SET,   RESET, RESET }, //
            9
33  };
34
35
36  void display7SEG(int num)
37  {
38      if (num < 0 || num > 9) return;
39
40      HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_SET);
41      HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_SET);
42
43      for (int i = 0; i < 7; i++)
44          HAL_GPIO_WritePin(segments[i].port, segments[i].pin,
                digitPatterns[num][i]);
45  }
46
47  int counter = 50;
48  int state = 0;
49
50  /* USER CODE END 0 */
```

- Now we can go for implementation:

Listing 2.2: Exercise 01 source code in HAL_TIM_PeriodElapsedCallback function

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    counter--;
    if(counter <= 0)
    {
        counter = 50;
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);


        if (state == 0)
        {
            display7SEG(1);
            HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin,
               GPIO_PIN_RESET);
            state = 1;
        }
        else
        {
            dis play7SEG(2);
            HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin,
               GPIO_PIN_RESET);
            state = 0;
        }
    }
}
```

Short question: What is the frequency of the scanning process?

- Each seven-segment LED stays on for $T_s = 0.5\,\text{s}$. With 2 LEDs, one full scan cycle (LED 1, then LED 2) takes $2 \times T_s = 1\,\text{s}$.

$$f = \frac{1}{2 \cdot T_s} = \frac{1}{2 \cdot 0.5} = 1\,\text{Hz}$$

- This matches the lab's formula $f = \frac{1}{N \cdot T_s}$, where N is the number of LEDs.

Therefore, the scanning frequency is 1 Hz.

2.2 Exercise 2

Extend to 4 seven segment LEDs and two LEDs (connected to PA4, labeled as **DOT**) in the middle as following:

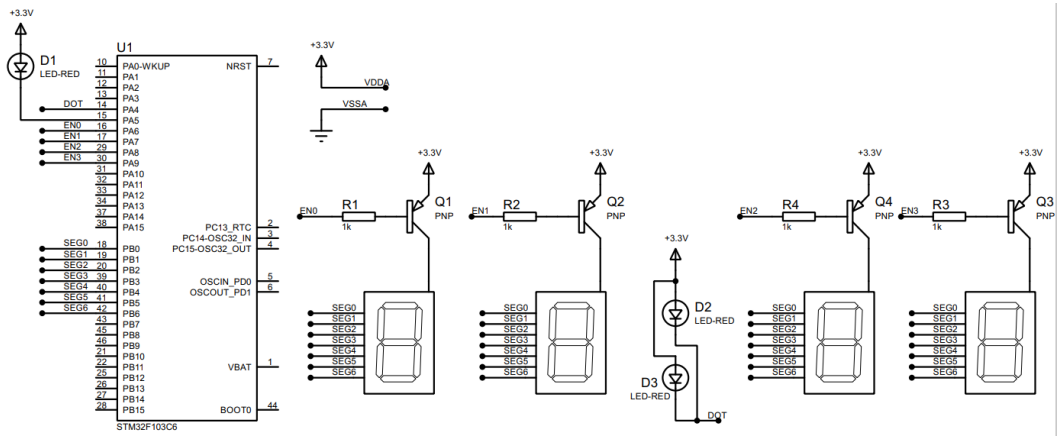


Figure 2.3: *Simulation schematic in Proteus*

Blink the two LEDs every second. Meanwhile, number 3 is displayed on the third seven segment and number 0 is displayed on the last one (to present 12 hour and a half). The switching time for each seven segment LED is also a half of second (500ms). **Implement your code in the timer interrupt function.**

Report 1: Capture your schematic from Proteus and show in the report.

Note: *As the exercise 09 and 10 add more components but it does not affect the schematic that we use for Exercise 3.2 to 3.7, we will submit the final schematic that can use all* `.hex` *files from 3.2 to 3.10*

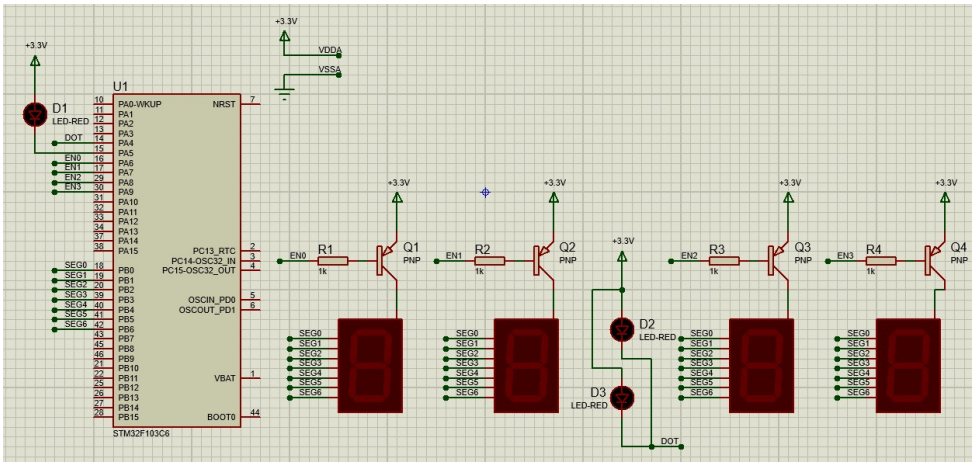


Figure 2.4: *Proteus schematic for four 7-segment LEDs*

Report 2: Present your source code in the **HAL_TIM_PeriodElapsedCallback** function.

- As the increasing of 7-segment LEDs, we will re-define some variables and update `void display7SEG(int num)`:

Listing 2.3: Initial implementation for Exercise 02

```
/* USER CODE BEGIN 0 */

// RE-USE Exercise 01 structures

void display7SEG(int num)
{
    if (num < 0 || num > 9) return;

    HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_SET);

    for (int i = 0; i < 7; i++)
        HAL_GPIO_WritePin(segments[i].port, segments[i].pin,
           digitPatterns[num][i]);
}


// From exercise 3.2
int digit_counter = 50;
int dot_counter   = 100;
int state = 0;
int digits[4] = {1, 2, 3, 0};

/* USER CODE END 0 */
```

Listing 2.4: Exercise 02 source code in HAL_TIM_PeriodElapsedCallback function

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    counter--;
    if(counter <= 0)
    {
        counter = 50;
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);


        if (state == 0)
        {
            display7SEG(1);
            HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin,
               GPIO_PIN_RESET);
            state = 1;
        }
        else
        {
            display7SEG(2);
            HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin,
               GPIO_PIN_RESET);
            state = 0;
        }
    }
}
```

Short question: What is the frequency of the scanning process?

- Applying the solve from Exercise 3.1, for Exercise 3.2 with 4 LEDs and $T_s = 0.5\,\text{s}$, it would be $\frac{1}{4 \cdot 0.5} = 0.5\,\text{Hz}$.

2.3 Exercise 3

Implement a function named **update7SEG(int index)**. An array of 4 integer numbers are declared in this case. The code skeleton in this exercise is presented as following:

Listing 2.5: An example for your source code

```
const int MAX_LED = 4;
int index_led = 0;
int led_buffer[4] = {1, 2, 3, 4};
void update7SEG(int index){
    switch (index){
        case 0:
            //Display the first 7SEG with led_buffer[0]
            break;
        case 1:
            //Display the second 7SEG with led_buffer[1]
            break;
        case 2:
            //Display the third 7SEG with led_buffer[2]
            break;
        case 3:
            //Display the forth 7SEG with led_buffer[3]
            break;
        default:
            break;
    }
}
```

This function should be invoked in the timer interrupt, e.g update7SEG(index_led++). The variable **index_led** is updated to stay in a valid range, which is from 0 to 3.

Report 1: Present the source code of the update7SEG function.

Listing 2.6: Source code of the update7SEG function.

```
void update7SEG(int index)
{
    switch (index)
```

```
    {
        case 0:
            display7SEG(led_buffer[0]);
            HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin,
                GPIO_PIN_RESET);
            break;
        case 1:
            display7SEG(led_buffer[1]);
            HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin,
                GPIO_PIN_RESET);
            break;
        case 2:
            display7SEG(led_buffer[2]);
            HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin,
                GPIO_PIN_RESET);
            break;
        case 3:
            display7SEG(led_buffer[3]);
            HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin,
                GPIO_PIN_RESET);
            break;
        default:
            break;
    }
}
```

Report 2: Present the source code in the HAL_TIM_PeriodElapsedCallback.

Listing 2.7: Source code in the HAL_TIM_PeriodElapsedCallback function.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    dot_counter--;
    if (dot_counter <= 0)
    {
        dot_counter = 100;
```

```
        HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
    }

    digit_counter--;
    if (digit_counter <= 0)
    {
        digit_counter = 50;
        update7SEG(index_led++);
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);


        if (index_led >= MAX_LED)
            index_led = 0;
    }
}
```

Students are proposed to change the values in the **led_buffer** array for unit test this function, which is used afterward.

- Changing `int led_buffer[4] = 1, 2, 3, 4;` into => `int led_buffer[4] = 6, 7, 6, 7;` works perfectly fine. We mention the two buffer in 2 files: `ex03_1.hex` and `ex03_2.hex`.

2.4 Exercise 4

Change the period of invoking update7SEG function in order to set the frequency of 4 seven segment LEDs to 1Hz. The DOT is still blinking every second.

- We already knew that the frequency of the scanning process from Exercise 3.2 is $f = 0.5\,\text{Hz}$. Using the formula $f = \frac{1}{N \cdot T_s}$ and applying $f = 1\,\text{Hz}$, we will get $T_s = 0.25\,\text{s} = 250\,\text{ms}$.

- With a 10 ms timer interrupt, $\frac{250\,\text{ms}}{10\,\text{ms}} = 25$. Therefore, we have to change `digit_counter = 25` in both `/* USER CODE BEGIN 0 */` and `void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeD *htim)`

Report 1: Present the source code in the **HAL_TIM_PeriodElapsedCallback**.

Listing 2.8: Source code in the HAL_TIM_PeriodElapsedCallback function.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    dot_counter--;
    if (dot_counter <= 0)
    {
        dot_counter = 100;
        HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
    }

    digit_counter--;
    if (digit_counter <= 0)
    {
        digit_counter = 25;
        update7SEG(index_led++);
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);


        if (index_led >= MAX_LED)
            index_led = 0;
    }
}
```

2.5 Exercise 5

Implement a digital clock with **hour** and **minute** information displayed by 2 seven segment LEDs. The code skeleton in the **main** function is presented as follows:

Listing 2.9: An example for your source code

```
int hour = 15, minute = 8, second = 50;

while(1){
    second++;
    if (second >= 60){
        second = 0;
        minute++;
```

```
    }
    if(minute >= 60){
        minute = 0;
        hour++;
    }
    if(hour >=24){
        hour = 0;
    }
    updateClockBuffer();
    HAL_Delay(1000);
}
```

The function **updateClockBuffer** will generate values for the array **led_buffer** according to the values of hour and minute. In the case these values are 1 digit number, digit 0 is added.

Report 1: Present the source code in the **updateClockBuffer** function.

Listing 2.10: Source code in the updateClockBuffer function.

```
void updateClockBuffer(void)
{
    led_buffer[0] = hour / 10;      // tens of hour
    led_buffer[1] = hour % 10;      // units of hour
    led_buffer[2] = minute / 10;    // tens of minute
    led_buffer[3] = minute % 10;    // units of minute
}
```

2.6 Exercise 6

The main target from this exercise to reduce the complexity (or reduce code processing) in the timer interrupt. The time consumed in the interrupt can lead to the nested interrupt issue, which can crash the whole system. A simple solution can disable the timer whenever the interrupt occurs, the enable it again. However, the real-time processing is not guaranteed anymore.

In this exercise, a software timer is created and its counter is count down every timer

interrupt is raised (every 10ms). By using this timer, the **Hal_Delay(1000)** in the main function is removed. In a MCU system, non-blocking delay is better than blocking delay. The details to create a software timer are presented bellow. The source code is added to your current program, **do not delete the source code you have on Exercise 5.**

Step 1: Declare variables and functions for a software timer, as following:

Listing 2.11: Software timer based timer interrupt

```
/* USER CODE BEGIN 0 */
int timer0_counter = 0;
int timer0_flag = 0;
int TIMER_CYCLE = 10;
void setTimer0(int duration){
  timer0_counter = duration /TIMER_CYCLE;
  timer0_flag = 0;
}
void timer_run(){
  if(timer0_counter > 0){
    timer0_counter--;
    if(timer0_counter == 0) timer0_flag = 1;
  }
}
/* USER CODE END 0 */
```

Please change the **TIMER_CYCLE** to your timer interrupt period. In the manual code above, it is **10ms**.

Step 2: The **timer_run()** is invoked in the timer interrupt as following:

Listing 2.12: Software timer based timer interrupt

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){

  timer_run();

  //YOUR OTHER CODE
}
```

Step 3: Use the timer in the main function by invoked setTimer0 function, then check for its flag (timer0_flag). An example to blink an LED connected to PA5 using software timer is shown as follows:

Listing 2.13: Software timer is used in main fuction to blink the LED

```
1 setTimer0(1000);
2 while (1){
3     if(timer0_flag == 1){
4         HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
5         setTimer0(2000);
6     }
7 }
```

Note: in order to stimulate successfully, make sure to not have `HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);` in `void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)`, since we using the same code from exercise 3.5 (which is also from exercise 3.4).

Report 1: if in line 1 of the code above is miss, what happens after that and why?

- The LED never blinks, or likely not toggle depending on its intial state.
- Reason: `timer0_counter` and `timer0_flag` both start at 0, and `timer_run()` does nothing while the counter is 0. The flag therefore never becomes 1, the if is never true, and `setTimer0(2000)` is never reached.

Report 2: if in line 1 of the code above is changed to setTimer0(1), what happens after that and why?

- The LED never blinks, same as Report 1.
- `timer0_counter = duration / TIMER_CYCLE;` = 1 / 10 = 0 because of integer division, so the counter starts at 0, `timer_run()` never decrements it, and the flag never gets set. Any duration smaller than `TIMER_CYCLE` (= 10 ms) rounds down to 0.

Report 3: if in line 1 of the code above is changed to setTimer0(10), what is changed compared to 2 first questions and why?

- The LED works.

- `timer0_counter` = 10 / 10 = 1, so after the next interrupt (10 ms) the counter reaches 0 and `timer0_flag` = 1. The LED toggles almost immediately, then `setTimer0(2000)` takes over, so it toggles every 2s afterwards. Compared with the first two cases, which never blink, this one blinks; compared with `setTimer0(1000)`, the only difference is that the first toggle happens after 10 ms instead of 1s.

2.7 Exercise 7

Upgrade the source code in Exercise 5 (update values for hour, minute and second) by using the software timer and remove the HAL_Delay function at the end. Moreover, the DOT (connected to PA4) of the digital clock is also moved to main function.

Report 1: Present your source code in the while loop on main function.

Listing 2.14: Source code in the while loop on main function.

```
while (1)
{
  if (timer0_flag == 1)
  {
    setTimer0(1000);

    second++;
    if (second >= 60) { second = 0; minute++; }
    if (minute >= 60) { minute = 0; hour++; }
    if (hour >= 24)   { hour = 0; }

    HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
  }
  updateClockBuffer();
}
```

2.8 Exercise 8

Move also the update7SEG() function from the interrupt timer to the main. Finally, the timer interrupt only used to handle software timers. All processing (or complex com-

putations) is move to an infinite loop on the main function, optimizing the complexity of the interrupt handler function.

Report 1: Present your source code in the the main function. In the case more extra functions are used (e.g. the second software timer), present them in the report as well.

- As we move `update7SEG()` to `int main(void)`, we have to add a second software time:

Listing 2.15: Updating global variables and functions

```
/* USER CODE BEGIN 0 */
//#####################################################
// From exercise 3.6 (to 3.10)

int timer0_counter = 0, timer0_flag = 0;   // DOT tick
int timer1_counter = 0, timer1_flag = 0;   // digit scanning (
   from 3.8)
int TIMER_CYCLE = 10;


void setTimer0(int duration) { timer0_counter = duration /
   TIMER_CYCLE; timer0_flag = 0; }
void setTimer1(int duration) { timer1_counter = duration /
   TIMER_CYCLE; timer1_flag = 0; }


void timer_run()
{
  if(timer0_counter > 0) { timer0_counter--; if(
     timer0_counter == 0) timer0_flag = 1; }
    if(timer1_counter > 0) { timer1_counter--; if(
       timer1_counter == 0) timer1_flag = 1; }
}
/* USER CODE END 0 */
```

- Then we remove everything from `void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)` but leave only `timer_run();`

Listing 2.16: Updating void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  // NO REMOVE FOR EXERCISE 3.6 (and later exercises)
  timer_run();
}
```

- Finally, we can implement the while loop.

Listing 2.17: Source code in the while loop on main function.)

```
while (1)
{
  if (timer0_flag == 1)
  {
    setTimer0(1000);

    second++;
    if (second >= 60) { second = 0; minute++; }
    if (minute >= 60) { minute = 0; hour++; }
    if (hour >= 24)   { hour = 0; }

    HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
  }
  updateClockBuffer();


    if (timer1_flag == 1)
    {
        setTimer1(250);

        update7SEG(index_led++);
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        if (index_led >= MAX_LED) index_led = 0;
    }
}
```

2.9 Exercise 9

This is an extra works for this lab. A LED Matrix is added to the system. A reference design is shown in figure bellow:

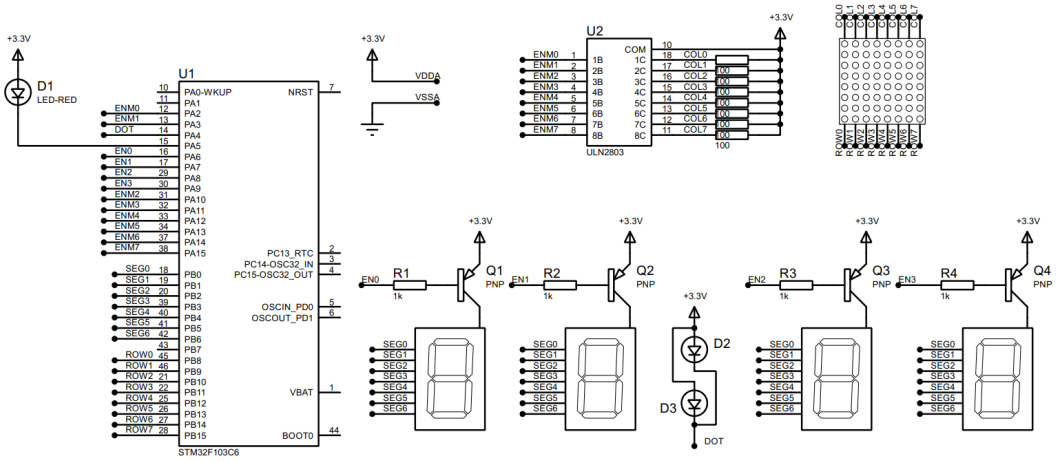


Figure 2.5: *LED matrix is added to the simulation*

In this schematic, two new components are added, including the **MATRIX-8X8-RED** and **ULN2803**, which is an NPN transistor array to enable the power supply for a column of the LED matrix. Students can change the enable signal (from ENM0 to ENM7) if needed. Finally, the data signal (from ROW0 to ROW7) is connected to PB8 to PB15.

Report 1: Present the schematic of your system by capturing the screen in Proteus.

Note: When we were building the schematic on Proteus, we found out that the "just-take-out" `MATRIX-8X8-RED` was upside down to the one in *Figure 2.5*. Therefore, we decided to change the wires to match with the default `MATRIX-8X8-RED` in Proteus.

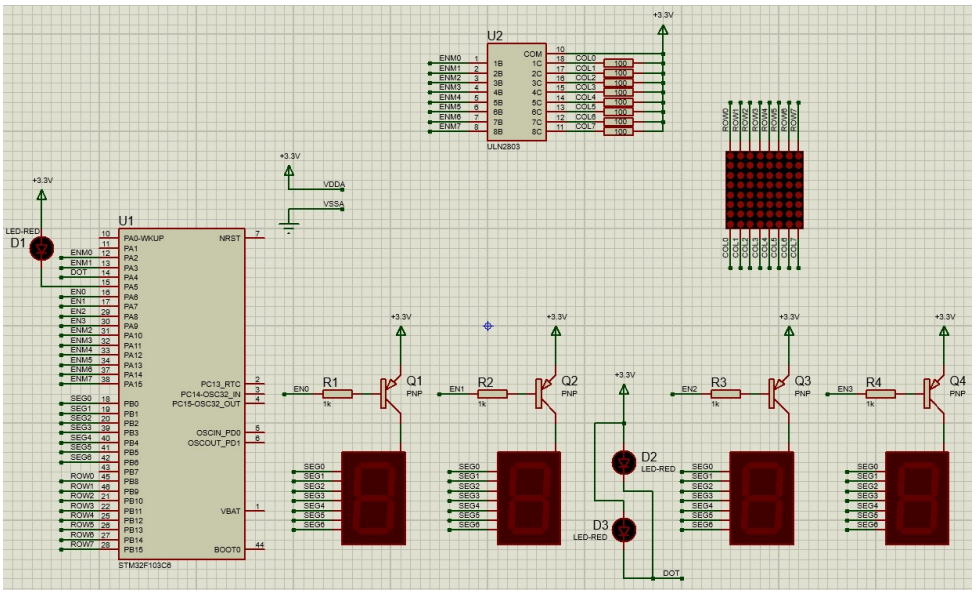


Figure 2.6: *Proteus schematic with MATRIX-8X8-RED and ULN2803*

Report 2: Implement the function, updateLEDMatrix(int index), which is similarly to 4 seven led segments.

Listing 2.18: Function to display data on LED Matrix

```
const int MAX_LED_MATRIX = 8;
int index_led_matrix = 0;
uint8_t matrix_buffer[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0
   x06, 0x07, 0x08};
void updateLEDMatrix(int index){
    switch (index){
        case 0:
            break;
        case 1:
            break;
        case 2:
            break;
        case 3:
            break;
        case 4:
            break;
        case 5:
            break;
```

```
18          case 6:
19              break;
20          case 7:
21              break;
22          default:
23              break;
24      }
25  }
```

Student are free to choose the invoking frequency of this function. However, this function is supposed to invoked in main function. Finally, please update the **matrix_buffer** to display character **"A"**.

- Before the full implementation, we will go through a brief about the component `MATRIX-8X8-RED` (which is implemented in *Figure 2.6*, not the example).

- After a few tests, we found out that `MATRIX-8X8-RED` interacted with `uint8_t matrix_buffer[8]` by converting hex value to binary base on the rows.

- For better illustration, we indicate the component by a 2x2 matrix, with "." states for 0 and "■" states for 1.

Table 2.1: Illustration of `MATRIX-8X8-RED`'s LEDs by a 2x2 matrix

	x0	x1	x2	x3	x4	x5	x6	x7
y0	.	.	.	.	.	.	.	.
y1	.	.	.	.	.	.	.	.
y2	.	.	.	.	.	.	.	.
y3	.	.	.	.	.	.	.	.
y4	.	.	.	.	.	.	.	.
y5	.	.	.	.	.	.	.	.
y6	.	.	.	.	.	.	.	.
y7	.	.	.	.	.	.	.	.

- Then we can display for the character A:

Table 2.2: Sample of a character A on MATRIX-8X8-RED

	x0	x1	x2	x3	x4	x5	x6	x7	Binary	HEX value
y0	.	.	.	■	■	.	.	.	00011000	0x18
y1	.	.	■	■	■	■	.	.	00111100	0x3C
y2	.	■	■	.	.	■	■	.	01100110	0x66
y3	■	■	.	.	.	.	■	■	11000011	0xC3
y4	■	■	.	.	.	.	■	■	11000011	0xC3
y5	■	■	■	■	■	■	■	■	11111111	0xFF
y6	■	■	.	.	.	.	■	■	11000011	0xC3
y7	■	■	.	.	.	.	■	■	11000011	0xC3

- For this exercise, we set the frequency $f = 12.5$Hz for the MATRIX-8X8-RED. For details, please check out `latex/main_code/ex09.txt`

Listing 2.19: Updates variables and functions for the software timer

```
//#################################################
// From exercise 3.6
int timer2_counter = 0, timer2_flag = 0;
void setTimer2(int duration) { timer2_counter = duration /
   TIMER_CYCLE; timer2_flag = 0; }

void timer_run()
{
  if(timer0_counter > 0) { timer0_counter--; if(
     timer0_counter == 0) timer0_flag = 1; }
    if(timer1_counter > 0) { timer1_counter--; if(
       timer1_counter == 0) timer1_flag = 1; }
    if(timer2_counter > 0) { timer2_counter--; if(
       timer2_counter == 0) timer2_flag = 1; }
}
```

Listing 2.20: Initialization for updateLEDMatrix(int index)

```
//#################################################
```

```
// From exercise 3.9

SegPin enm[8] =
{
    {ENM0_GPIO_Port, ENM0_Pin}, {ENM1_GPIO_Port, ENM1_Pin},
    {ENM2_GPIO_Port, ENM2_Pin}, {ENM3_GPIO_Port, ENM3_Pin},
    {ENM4_GPIO_Port, ENM4_Pin}, {ENM5_GPIO_Port, ENM5_Pin},
    {ENM6_GPIO_Port, ENM6_Pin}, {ENM7_GPIO_Port, ENM7_Pin},
};


SegPin row[8] =
{
    {ROW0_GPIO_Port, ROW0_Pin}, {ROW1_GPIO_Port, ROW1_Pin},
    {ROW2_GPIO_Port, ROW2_Pin}, {ROW3_GPIO_Port, ROW3_Pin},
    {ROW4_GPIO_Port, ROW4_Pin}, {ROW5_GPIO_Port, ROW5_Pin},
    {ROW6_GPIO_Port, ROW6_Pin}, {ROW7_GPIO_Port, ROW7_Pin},
};


void displayRow(uint8_t pattern)
{
    HAL_GPIO_WritePin(ENM0_GPIO_Port, ENM0_Pin,
       GPIO_PIN_RESET);
    HAL_GPIO_WritePin(ENM1_GPIO_Port, ENM1_Pin,
       GPIO_PIN_RESET);
    HAL_GPIO_WritePin(ENM2_GPIO_Port, ENM2_Pin,
       GPIO_PIN_RESET);
    HAL_GPIO_WritePin(ENM3_GPIO_Port, ENM3_Pin,
       GPIO_PIN_RESET);
    HAL_GPIO_WritePin(ENM4_GPIO_Port, ENM4_Pin,
       GPIO_PIN_RESET);
    HAL_GPIO_WritePin(ENM5_GPIO_Port, ENM5_Pin,
       GPIO_PIN_RESET);
    HAL_GPIO_WritePin(ENM6_GPIO_Port, ENM6_Pin,
```

```
        GPIO_PIN_RESET);
31      HAL_GPIO_WritePin(ENM7_GPIO_Port, ENM7_Pin,
        GPIO_PIN_RESET);
32
33      for (int r = 0; r < 8; r++)
34          HAL_GPIO_WritePin(row[r].port, row[r].pin, ((pattern
            >> r) & 1) ? GPIO_PIN_SET : GPIO_PIN_RESET);
35  }
36
37
38  // ================================================
```

Listing 2.21: Implementation of `updateLEDMatrix()` for displaying 'A'

```
1   // The implementation also includes the adjustment of matrix\
    _buffer[8] to display the character 'A'.
2   const int MAX_LED_MATRIX = 8;
3   int index_led_matrix = 0;
4   uint8_t matrix_buffer[8] = {0x18, 0x3C, 0x66, 0xC3, 0xC3, 0
    xFF, 0xC3, 0xC3};
5   // Exercise example:    uint8_t matrix_buffer[8] = {0x01, 0
    x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};
6   // For display letter A:  uint8_t matrix_buffer[8] = {0x18, 0
    x3C, 0x66, 0xC3, 0xC3, 0xFF, 0xC3, 0xC3};
7
8   void updateLEDMatrix(int index)
9   {
10    switch (index)
11    {
12      case 0:
13        displayRow(matrix_buffer[0]);
14        HAL_GPIO_WritePin(ENM0_GPIO_Port, ENM0_Pin,
            GPIO_PIN_SET);
15        break;
16      case 1:
17        displayRow(matrix_buffer[1]);
```

```
18      HAL_GPIO_WritePin(ENM1_GPIO_Port, ENM1_Pin,
            GPIO_PIN_SET);
19      break;
20    case 2:
21      displayRow(matrix_buffer[2]);
22      HAL_GPIO_WritePin(ENM2_GPIO_Port, ENM2_Pin,
            GPIO_PIN_SET);
23      break;
24    case 3:
25      displayRow(matrix_buffer[3]);
26      HAL_GPIO_WritePin(ENM3_GPIO_Port, ENM3_Pin,
            GPIO_PIN_SET);
27      break;
28    case 4:
29      displayRow(matrix_buffer[4]);
30      HAL_GPIO_WritePin(ENM4_GPIO_Port, ENM4_Pin,
            GPIO_PIN_SET);
31      break;
32    case 5:
33      displayRow(matrix_buffer[5]);
34      HAL_GPIO_WritePin(ENM5_GPIO_Port, ENM5_Pin,
            GPIO_PIN_SET);
35      break;
36    case 6:
37      displayRow(matrix_buffer[6]);
38      HAL_GPIO_WritePin(ENM6_GPIO_Port, ENM6_Pin,
            GPIO_PIN_SET);
39      break;
40    case 7:
41      displayRow(matrix_buffer[7]);
42      HAL_GPIO_WritePin(ENM7_GPIO_Port, ENM7_Pin,
            GPIO_PIN_SET);
43      break;
44    default:
45      break;
```

```
46   }
47 }
```

- Finally, with all of that, we just need to call in `int main(void)`.

Listing 2.22: Implementation for while loop

```
1  /* USER CODE BEGIN 2 */
2  HAL_TIM_Base_Start_IT(&htim2);
3  setTimer0(1000);
4  setTimer1(250); // add from 3.8
5  setTimer2(10);  // add from 3.9
6
7
8  while (1)
9  {
10   if (timer0_flag == 1)
11   {
12     setTimer0(1000);
13
14     second++;
15     if (second >= 60) { second = 0; minute++; }
16     if (minute >= 60) { minute = 0; hour++; }
17     if (hour >= 24)   { hour = 0; }
18
19     HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
20   }
21   updateClockBuffer();
22
23
24     if (timer1_flag == 1)
25     {
26         setTimer1(250);
27
28         update7SEG(index_led++);
29         HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
30         if (index_led >= MAX_LED) index_led = 0;
```

```
31      }
32
33
34      if (timer2_flag == 1)
35      {
36          setTimer2(10);
37          updateLEDMatrix(index_led_matrix++);
38          if (index_led_matrix >= MAX_LED_MATRIX)
               index_led_matrix = 0;
39      }
40  }
```

2.10 Exercise 10

Create an animation on LED matrix, for example, the character is shifted to the left.

Report 1: Briefly describe your solution and present your source code in the report.

- For this exercise, we came to a solution is to make the character shift to the left.

- In order to this, we can use a way is to shift bits, since the matrix LEDs work as binaries. We will take off the left-most column -> shift the whole matrix to the left -> put the take-out to the right-most.

Listing 2.23: Implementation for the shift bits function

```
1 void shiftLeft(void)
2 {
3     for (int i = 0; i < MAX_LED_MATRIX; i++)
4     {
5         uint8_t b = matrix_buffer[i];
6         matrix_buffer[i] = (b >> 1) | (b << 7);
7     }
8 }
```

- In the loop:

- `b » 1` does the "move left" part.
- `b « 7` rescues the cell that would have been lost and puts it on the far right.

- | merges the two results into one row.

so the whole `matrix_buffer[i] = (b >> 1) | (b << 7);` is just updating the value from the combanation of two different results to make like it is shifting at row `i`.

- Then we just need to make a new software timer.

(*or you can just synchronize this one with the scanning 7-segment LEDs one if you are lazy*)

Listing 2.24: Updates variables and functions for the software timer

```
//#####################################################
// From exercise 3.6
int timer3_counter = 0, timer3_flag = 0;
void setTimer3(int duration) { timer3_counter = duration /
   TIMER_CYCLE; timer3_flag = 0; }

void timer_run()
{
  if(timer0_counter > 0) { timer0_counter--; if(
     timer0_counter == 0) timer0_flag = 1; }
    if(timer1_counter > 0) { timer1_counter--; if(
       timer1_counter == 0) timer1_flag = 1; }
    if(timer2_counter > 0) { timer2_counter--; if(
       timer2_counter == 0) timer2_flag = 1; }
    if(timer3_counter > 0) { timer3_counter--; if(
       timer3_counter == 0) timer3_flag = 1; }
}
```

Listing 2.25: Implementation for while loop

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim2);
setTimer0(1000);
setTimer1(250); // add from 3.8
setTimer2(10);  // add from 3.9
setTimer2(200); // add from 3.10

while (1)
```

```
{
    /*Old 3.9 code*/


    if (timer3_flag == 1)
    {
        setTimer3(200);
        shiftLeft();
    }
}
```